

Smart Funkos — Documento Técnico Inicial

1. Decisão de arquitetura

A Smart Funkos terá uma arquitetura centralizada, onde o backend é a fonte única da verdade para site, app mobile e painel administrativo.

A stack inicial será:

```
Web / Site público: Next.js  
Admin: Next.js  
Backend/API: Next.js Route Handlers  
Mobile: Flutter  
Banco: Supabase Postgres  
Auth: Supabase Auth  
Storage: Supabase Storage  
Realtime: Supabase Realtime  
Funções críticas: Next.js API ou Supabase Edge Functions
```

O objetivo é evitar múltiplos sistemas com dados diferentes.

Não deve existir:

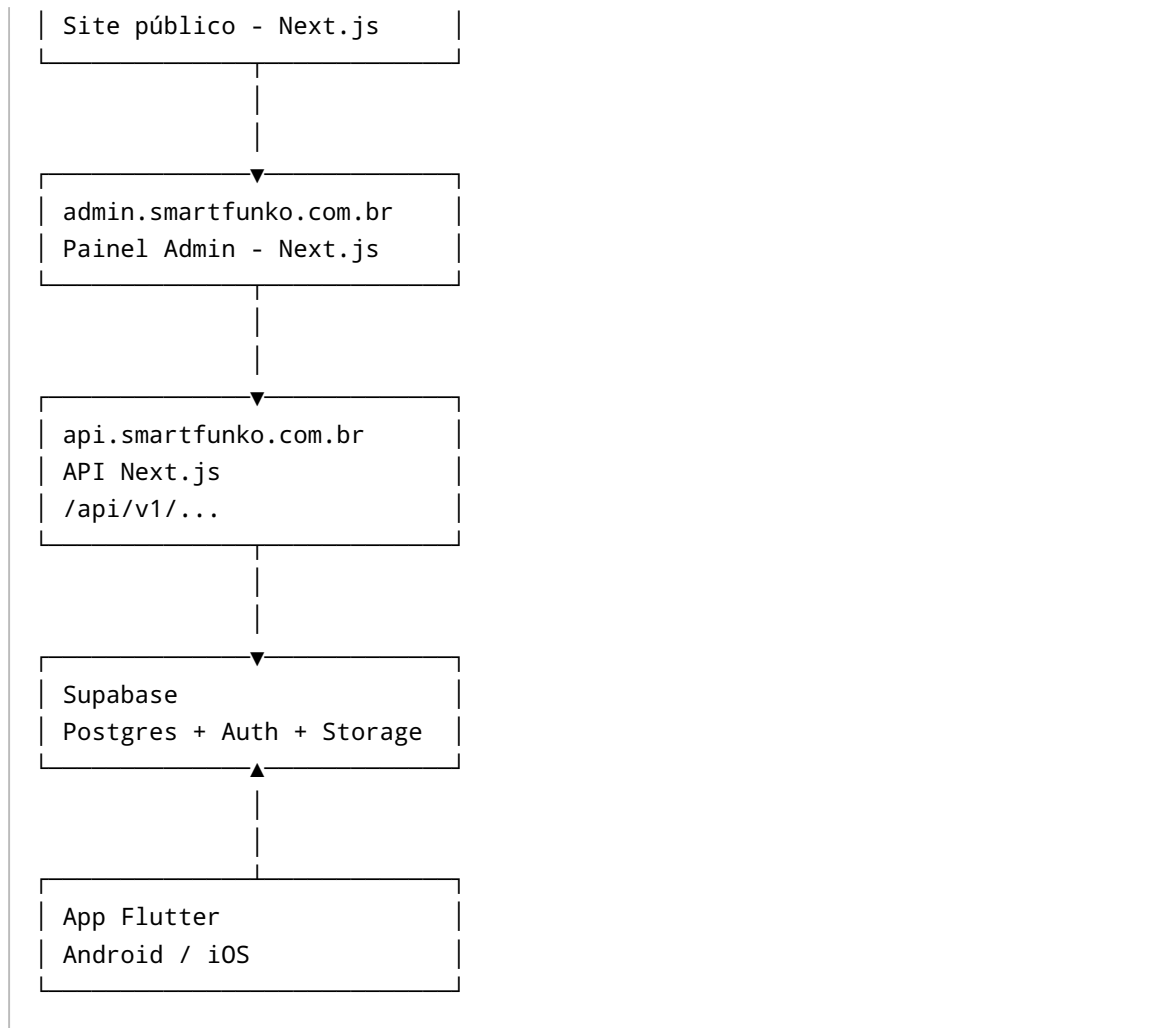
```
site com catálogo próprio  
app com catálogo próprio  
WhatsApp como controle oficial  
planilha como caixa oficial  
Base44 como fonte final de dados
```

Deve existir:

```
um banco central  
uma API central  
um modelo único de pedidos  
um controle único de estoque  
um controle único de pagamentos  
um controle único de caixa
```

2. Visão geral da arquitetura

```
| smartfunktoko.com.br |
```



O site, o admin e o app Flutter devem consumir a mesma API e trabalhar com o mesmo banco.

3. Responsabilidade de cada parte

3.1 Next.js

O Next.js será usado para:

- site público
- catálogo público
- páginas institucionais
- SEO
- painel administrativo
- API backend
- autenticação web
- regras de negócio server-side
- integração com pagamentos
- integração com webhooks

3.2 Flutter

O Flutter será usado para:

- app mobile Android
- app mobile iOS
- área do cliente
- meus pedidos
- wishlist
- notificações
- leilões no futuro
- status de pedido
- experiência mobile nativa

3.3 Supabase

O Supabase será usado para:

- banco Postgres
- autenticação
- controle de sessão
- storage de imagens
- realtime
- políticas de segurança
- backup/exportação

4. Regra principal de arquitetura

O Flutter não deve acessar livremente o banco para executar regras sensíveis.

O site e o app podem ler dados públicos e dados próprios do usuário, mas operações críticas devem passar pela API.

Operações críticas:

- criar pedido
- reservar estoque
- registrar pagamento
- cancelar pedido
- alterar status
- aceitar lance
- fechar leilão
- alterar preço

- aplicar desconto
- gerar caixa

Essas operações devem ficar no backend.

5. Estrutura de domínios

Estrutura recomendada:

```
smartfunktoko.com.br
Site público

admin.smartfunktoko.com.br
Painel administrativo

api.smartfunktoko.com.br
API para Flutter, site e admin

app.smartfunktoko.com.br
Área web do cliente, se necessário
```

No início, é possível simplificar:

```
smartfunktoko.com.br
Site público

smartfunktoko.com.br/admin
Painel admin

smartfunktoko.com.br/api/v1
API

Flutter App
Consume smartfunktoko.com.br/api/v1
```

A separação em subdomínios pode vir depois.

6. Estrutura de repositório

Recomendação inicial usando monorepo:

```
smartfunktoko/
  apps/
  web/
```

```
# Next.js: site, admin e API

mobile/
  # Flutter app

packages/
  shared/
    # contratos, tipos, constantes, documentação de API

supabase/
  migrations/
  seed/
  policies/
  functions/

docs/
  architecture.md
  database.md
  api.md
  business-rules.md
  roadmap.md
```

Caso queira simplificar no início:

```
smartfunktio/
  web/
    # Next.js

  mobile/
    # Flutter

  supabase/
    # migrations e políticas

  docs/
    # documentação
```

7. Estrutura do projeto Next.js

```
web/
  src/
    app/
      page.tsx
      catalogo/
      produto/[slug]/
      como-funciona/
      admin/
```

```
    dashboard/  
    clientes/  
    produtos/  
    pedidos/  
    estoque/  
    caixa/  
    lotes/  
  api/  
    v1/  
      auth/  
      customers/  
      products/  
      orders/  
      payments/  
      inventory/  
      wishlist/  
      purchase-batches/  
      notifications/  
  
  components/  
    ui/  
    layout/  
    product/  
    order/  
    admin/  
  
  lib/  
    supabase/  
      client.ts  
      server.ts  
      admin.ts  
    auth/  
    validations/  
    permissions/  
    money.ts  
    dates.ts  
  
  modules/  
    customers/  
    products/  
    orders/  
    payments/  
    inventory/  
    cashflow/  
    wishlist/  
    purchase-batches/  
    notifications/  
  
  types/  
    database.ts  
    api.ts
```

8. Estrutura do app Flutter

```
mobile/  
  lib/  
    main.dart  
  
  app/  
    app.dart  
    router.dart  
    theme.dart  
  
  core/  
    config/  
    http/  
    auth/  
    errors/  
    utils/  
  
  features/  
    auth/  
      data/  
      domain/  
      presentation/  
  
    catalog/  
      data/  
      domain/  
      presentation/  
  
    product_detail/  
      data/  
      domain/  
      presentation/  
  
    wishlist/  
      data/  
      domain/  
      presentation/  
  
    orders/  
      data/  
      domain/  
      presentation/  
  
    payments/  
      data/  
      domain/  
      presentation/
```

```
profile/  
  data/  
  domain/  
  presentation/  
  
notifications/  
  data/  
  domain/  
  presentation/  
  
auctions/  
  data/  
  domain/  
  presentation/  
  
shared/  
  widgets/  
  models/  
  services/
```

9. Modelo de dados inicial

9.1 profiles

Representa usuários autenticados.

```
profiles  
- id  
- auth_user_id  
- name  
- email  
- phone  
- role  
- avatar_url  
- created_at  
- updated_at
```

Roles iniciais:

```
customer  
staff  
admin  
owner
```


9.2 customers

Representa compradores.

```
customers
- id
- profile_id
- name
- email
- phone
- cpf
- instagram
- status
- notes
- created_at
- updated_at
```

Status:

```
active
vip
blocked
```

9.3 addresses

```
addresses
- id
- customer_id
- zipcode
- street
- number
- complement
- neighborhood
- city
- state
- is_default
- created_at
- updated_at
```

9.4 franchises

```
franchises
- id
```

```
- name
- slug
- image_url
- status
- created_at
- updated_at
```

Exemplos:

```
One Piece
Marvel
DC
Disney
Naruto
Black Clover
Harry Potter
Star Wars
```

9.5 products

```
products
- id
- name
- slug
- franchise_id
- funko_number
- description
- main_image_url
- status
- created_at
- updated_at
```

Status:

```
active
inactive
archived
```

9.6 product_variants

```
product_variants
- id
- product_id
```

- sku
- condition
- type
- source
- sale_price
- market_price
- estimated_cost
- status
- created_at
- updated_at

Condition:

new
used
damaged_box

Type:

common
exclusive
chase
glow
special

Source:

own_stock
national
international
preorder

Status:

available
order_only
preorder
sold_out
hidden

9.7 product_images

```
product_images
- id
- product_id
- image_url
- sort_order
- created_at
```

9.8 inventory_items

Controle por unidade.

```
inventory_items
- id
- product_variant_id
- sku
- status
- location
- purchase_cost
- landed_cost
- reserved_for_order_item_id
- notes
- created_at
- updated_at
```

Status:

```
available
reserved
sold
in_transit
damaged
unavailable
```

9.9 orders

```
orders
- id
- order_number
- customer_id
- channel
- status
```

```
- subtotal
- discount
- shipping_amount
- total
- notes
- internal_notes
- created_by
- created_at
- updated_at
```

Channel:

```
whatsapp
website
app
admin
auction
preorder
```

Status:

```
draft
pending_payment
partially_paid
paid
processing
ready_to_ship
shipped
delivered
cancelled
refunded
```

9.10 order_items

```
order_items
- id
- order_id
- product_variant_id
- inventory_item_id
- quantity
- unit_price
- total_price
- cost_estimate
- final_cost
- source
- status
```

- created_at
- updated_at

Source:

```
stock
national_order
international_order
preorder
auction
```

Status:

```
requested
reserved
waiting_payment
paid
waiting_purchase
purchased
in_transit
received
ready_to_ship
shipped
delivered
cancelled
```

Regra importante:

O pedido tem status geral.
Cada item também tem status próprio.

9.11 payments

```
payments
- id
- order_id
- customer_id
- method
- provider
- provider_payment_id
- amount
- fee_amount
- net_amount
- status
```

```
- paid_at
- created_by
- created_at
```

Method:

```
pix
credit_card
debit_card
cash
manual
```

Status:

```
pending
paid
failed
cancelled
refunded
```

9.12 cash_entries

```
cash_entries
- id
- type
- category
- order_id
- payment_id
- amount
- description
- occurred_at
- created_by
- created_at
```

Type:

```
income
expense
adjustment
```

Category:

```
sale
supplier_purchase
shipping
payment_fee
refund
manual_adjustment
```

9.13 wishlist_items

```
wishlist_items
- id
- customer_id
- product_id
- desired_price
- priority
- notes
- created_at
```

Priority:

```
low
medium
high
```

9.14 purchase_batches

```
purchase_batches
- id
- name
- type
- status
- closing_date
- supplier_name
- estimated_total_cost
- final_total_cost
- notes
- created_at
- updated_at
```

Type:


```
national
international
```

Status:

```
open
closed
ordered
in_transit
received
completed
cancelled
```

9.15 purchase_batch_items

```
purchase_batch_items
- id
- purchase_batch_id
- order_item_id
- product_variant_id
- quantity
- supplier_cost
- shipping_cost_share
- tax_cost_share
- final_landed_cost
- status
- created_at
- updated_at
```

Status:

```
pending
ordered
shipped
received
cancelled
```

9.16 notifications

```
notifications
- id
- customer_id
```

```
- title
- message
- type
- read_at
- created_at
```

Type:

```
order_update
payment
wishlist
auction
general
```

9.17 admin_action_logs

```
admin_action_logs
- id
- admin_id
- action
- entity_type
- entity_id
- old_value
- new_value
- created_at
```

Esse log deve registrar ações administrativas relevantes.

Exemplos:

```
criou pedido
editou valor
aplicou desconto
marcou como pago
cancelou pedido
alterou estoque
bloqueou cliente
criou lote de compra
```

10. API inicial

A API deve seguir uma estrutura versionada:

/api/v1

10.1 Auth

POST /api/v1/auth/session
GET /api/v1/auth/me
POST /api/v1/auth/logout

10.2 Customers

GET /api/v1/customers
GET /api/v1/customers/:id
POST /api/v1/customers
PATCH /api/v1/customers/:id
PATCH /api/v1/customers/:id/block
PATCH /api/v1/customers/:id/vip

10.3 Products

GET /api/v1/products
GET /api/v1/products/:id
GET /api/v1/products/slug/:slug
POST /api/v1/products
PATCH /api/v1/products/:id
DELETE /api/v1/products/:id

10.4 Product variants

GET /api/v1/products/:productId/variants
POST /api/v1/products/:productId/variants
PATCH /api/v1/product-variants/:id

10.5 Inventory

```
GET /api/v1/inventory
POST /api/v1/inventory
POST /api/v1/inventory/:id/reserve
POST /api/v1/inventory/:id/release
POST /api/v1/inventory/:id/mark-sold
POST /api/v1/inventory/:id/mark-damaged
```

10.6 Orders

```
GET /api/v1/orders
GET /api/v1/orders/:id
POST /api/v1/orders
PATCH /api/v1/orders/:id
POST /api/v1/orders/:id/cancel
GET /api/v1/orders/:id/items
POST /api/v1/orders/:id/items
PATCH /api/v1/order-items/:id/status
```

10.7 Customer account

```
GET /api/v1/me
GET /api/v1/me/orders
GET /api/v1/me/orders/:id
GET /api/v1/me/payments
GET /api/v1/me/wishlist
POST /api/v1/me/wishlist
DELETE /api/v1/me/wishlist/:id
GET /api/v1/me/notifications
PATCH /api/v1/me/notifications/:id/read
```

10.8 Payments

```
GET /api/v1/payments
POST /api/v1/payments/manual
POST /api/v1/payments/pix
POST /api/v1/payments/webhook
POST /api/v1/payments/:id/refund
```

No MVP, o principal endpoint será:

```
POST /api/v1/payments/manual
```

Pix automático pode entrar na segunda fase.

10.9 Cashflow

```
GET /api/v1/cashflow
POST /api/v1/cashflow/manual-entry
GET /api/v1/cashflow/summary
GET /api/v1/cashflow/pending
```

10.10 Wishlist

```
GET /api/v1/wishlist
POST /api/v1/wishlist
DELETE /api/v1/wishlist/:id
GET /api/v1/admin/wishlist/demand
```

10.11 Purchase batches

```
GET /api/v1/purchase-batches
GET /api/v1/purchase-batches/:id
POST /api/v1/purchase-batches
PATCH /api/v1/purchase-batches/:id
POST /api/v1/purchase-batches/:id/items
POST /api/v1/purchase-batches/:id/close
POST /api/v1/purchase-batches/:id/mark-ordered
POST /api/v1/purchase-batches/:id/mark-received
```

10.12 Dashboard

```
GET /api/v1/admin/dashboard
```

Deve retornar:

```
vendas do dia
recebido no dia
```

pendente a receber
pedidos novos
pedidos aguardando pagamento
pedidos em trânsito
produtos em estoque
produtos reservados
top desejos

11. Regras de negócio

11.1 Pedido

- Todo pedido deve ter cliente.
- Todo pedido deve ter pelo menos um item.
- Pedido pode ter pagamento parcial.
- Pedido pode nascer do WhatsApp, site, app ou admin.
- Pedido cancelado deve liberar estoque reservado.
- Pedido pago deve gerar entrada no caixa.

11.2 Item do pedido

- Cada item tem status próprio.
- Item de pronta-entrega deve reservar estoque.
- Item internacional deve poder entrar em lote de compra.
- Item cancelado deve sair do lote ou liberar estoque.

11.3 Estoque

- Produto reservado não aparece como disponível.
- Produto vendido não pode ser vendido novamente.
- Toda alteração manual de estoque deve gerar log.
- Estoque deve ser controlado por unidade sempre que possível.

11.4 Pagamento

- Pagamento confirmado gera cash_entry.
- Pagamento parcial deixa pedido parcialmente pago.
- Pagamento total deixa pedido pago.

- Estorno gera lançamento negativo no caixa.
- Pagamento manual exige admin ou owner.

11.5 Cliente

- Cliente só pode ver os próprios pedidos.
- Cliente bloqueado não pode comprar ou participar de leilão.
- Admin pode consultar clientes.
- Owner pode exportar dados.

11.6 Lote internacional

- Lote aberto aceita itens.
- Lote fechado não aceita novos itens sem permissão.
- Item recebido no lote atualiza o status do order_item.
- Custo final deve atualizar margem do item.

12. Segurança

Regras obrigatórias:

- RLS habilitado no Supabase.
- Service role somente no servidor.
- Flutter nunca deve receber service role key.
- Admin deve ter controle por role.
- Cliente só vê dados próprios.
- Ações críticas passam pela API.
- Toda alteração financeira gera log.
- Toda alteração de estoque gera log.
- Webhooks devem validar assinatura do provedor.

13. Estratégia de autenticação

13.1 Web/Admin

O admin usa Supabase Auth via Next.js.

Fluxo:

```
Admin faz login
Next.js valida sessão
Sistema lê role do usuário
Permissões são aplicadas
```

13.2 Flutter

O Flutter usa Supabase Auth para login.

Fluxo:

```
Cliente faz login no app
App recebe sessão/JWT
App chama API Next.js com token
API valida token
API retorna apenas dados autorizados
```

14. Fluxos principais

14.1 Pedido vindo do WhatsApp

```
Cliente pede no WhatsApp
Admin cria cliente, se não existir
Admin cria pedido no painel
Admin adiciona itens
Sistema calcula total
Sistema registra pedido
Cliente recebe link para acompanhar/pagar
Pagamento é registrado
Pedido muda de status
Caixa é atualizado
```

14.2 Pedido pelo site

```
Cliente acessa produto
Adiciona ao carrinho
Sistema valida estoque
Cliente cria login ou identifica telefone
Sistema cria pedido
Cliente paga
Pagamento confirmado
```


Estoque é baixado
Pedido aparece no admin

14.3 Pedido internacional

Cliente solicita produto
Admin aprova preço
Pedido é criado
Cliente paga sinal ou total
Item entra em lote internacional
Lote é fechado
Admin compra os produtos
Lote entra em trânsito
Produto chega
Item muda para recebido
Cliente é notificado

14.4 Área do cliente

Cliente abre app Flutter
Vê pedidos
Vê status por item
Vê pendências
Vê wishlist
Recebe notificações
Atualiza perfil

15. MVP inicial

O MVP deve conter:

Auth
Roles
Clientes
Produtos
Variantes
Imagens
Estoque básico
Pedidos
Itens de pedido
Pagamentos manuais
Caixa básico

- Wishlist
- Lotes de compra
- Notificações internas
- Logs administrativos
- Dashboard admin
- Área cliente no app

O MVP não precisa conter:

- leilão em tempo real
- Pix automático
- cartão automático
- nota fiscal
- frete automático
- app publicado nas lojas
- rifas/campanhas
- BI avançado

16. Roadmap de implementação

Sprint 1 — Fundação

- Criar projeto Supabase
- Criar projeto Next.js
- Criar projeto Flutter
- Configurar ambiente
- Configurar Auth
- Criar migrations iniciais
- Criar roles
- Criar RLS básico
- Criar storage de imagens

Sprint 2 — Produtos e clientes

- CRUD de clientes
- CRUD de franquias
- CRUD de produtos
- CRUD de variantes
- Upload de imagens
- Catálogo público

Sprint 3 — Pedidos

- Criar pedido manual
- Adicionar item ao pedido
- Status de pedido
- Status de item
- Página de detalhe do pedido
- Conta do cliente

Sprint 4 — Pagamentos e caixa

- Registrar pagamento manual
- Pagamento parcial
- Pagamento total
- Entrada de caixa automática
- Relatório de pendências
- Relatório de recebidos
- Dashboard simples

Sprint 5 — Estoque

- Adicionar unidade ao estoque
- Reservar estoque
- Liberar reserva
- Marcar como vendido
- Ver estoque disponível
- Ver estoque reservado

Sprint 6 — App Flutter MVP

- Login
- Home
- Catálogo
- Detalhe do produto
- Wishlist
- Meus pedidos
- Detalhe do pedido
- Minhas pendências
- Perfil
- Notificações

Sprint 7 — Lotes de compra

```
Criar lote nacional/internacional
Adicionar itens ao lote
Fechar lote
Marcar lote como comprado
Marcar lote como recebido
Atualizar status dos pedidos
```

Sprint 8 — Preparação para pagamentos automáticos

```
Escolher gateway
Criar endpoint de Pix
Criar endpoint de webhook
Criar logs de webhook
Testar baixa automática
```

17. Critério de sucesso do MVP

O MVP será considerado pronto quando:

- Todo pedido novo puder ser registrado no sistema.
- Todo cliente conseguir ver seus pedidos.
- Todo pagamento manual puder ser registrado.
- Toda entrada de pagamento gerar caixa.
- Todo produto vendido ou reservado afetar estoque.
- Os donos conseguirem ver vendas, recebidos e pendências.
- Itens internacionais puderem entrar em lote.
- Wishlist gerar visão de demanda.
- WhatsApp deixar de ser o controle oficial da operação.

18. Decisão final

A Smart Funkos deve seguir com:

```
Next.js
para site, admin e backend/API

Flutter
para app mobile

Supabase
```

para banco, auth, storage e realtime

Postgres
como fonte única da verdade

O foco inicial não deve ser criar um e-commerce bonito.

O foco inicial deve ser criar o core operacional:

cliente
produto
pedido
pagamento
estoque
caixa
wishlist
lote de compra

Depois disso, o e-commerce, o app nas lojas, leilões e automações financeiras entram como evolução natural.